

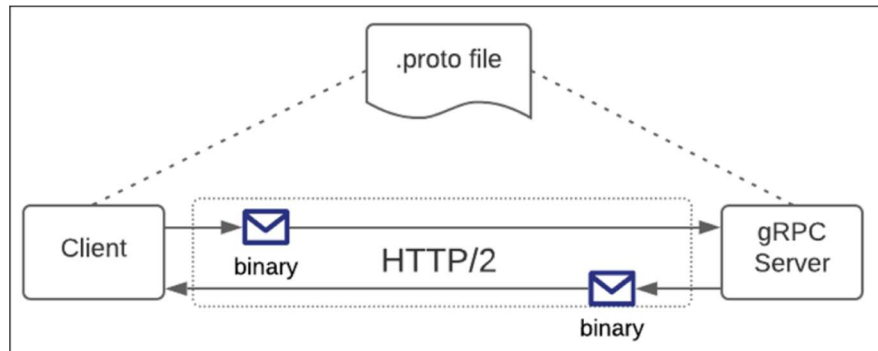


Duration: 2 Lab Session.

Individual work.

Grading: Executing and show understanding of the code.

Part1 : Building API with Node.js and gRPC



Description:

Part1 aims at implementing a simple application using Node.js and gRPC.

The server provides the following operations:

- Get all students: getAllStudents.
- Add a new student: addStudent.
- Get a student by its id: getStudentbyId.

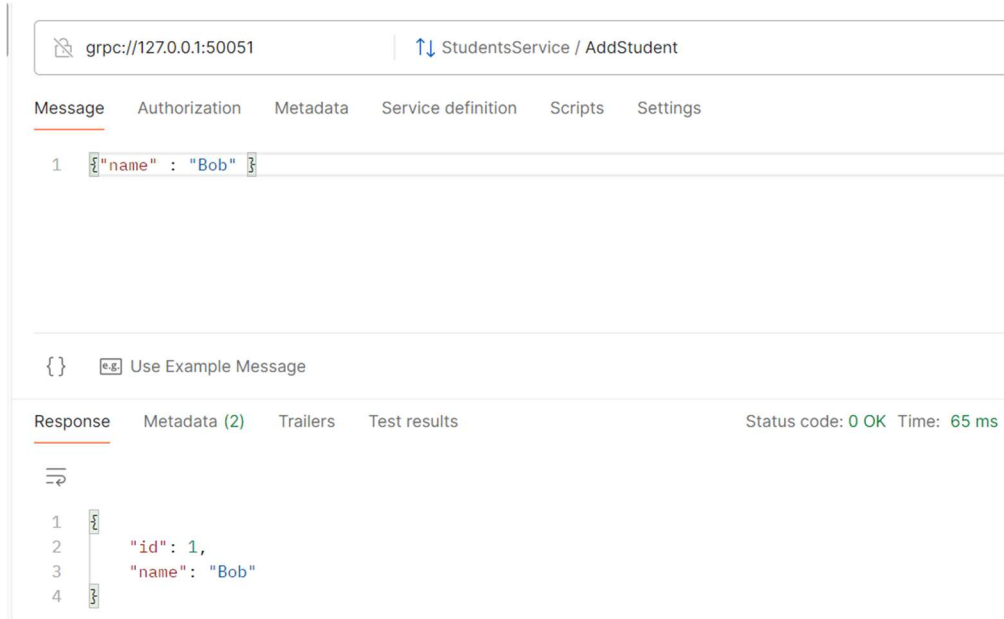
Step 1: Complete the following protocol buffer file.

```
syntax = 'proto3';
package StudentsServicePackage;
// Our Book service definition.
service StudentsService
{
    rpc createStudent (Student) returns (Student);
    rpc getStudentbyId (StudentRequest) returns (Student);
    rpc getAllStudents (Empty) returns (StudentsList);
}
// Our student message definition.
message Student {
    int32 id = 1;
    string name = 2;
}
...
```

Step2: Complete the server code.

```
const grpc = require('@grpc/grpc-js');
const protoLoader = require('@grpc/proto-loader');
const packageDefinition = protoLoader.loadSync('students.proto', {});
const StudentsServicePackage =
  grpc.loadPackageDefinition(packageDefinition).StudentsServicePackage;
// Create a server
const server = new grpc.Server();
// Add the service
server.addService(StudentsServicePackage.StudentsService.service, {
  'addStudent': addStudent,
  'GetStudentByID': GetStudentByID,
  ...
});
...
```

Step3: Test the service using postman as illustrated in the following figures.



grpc://127.0.0.1:50051

↕ StudentsService / GetStudentByID

Message

Authorization

Metadata

Service definition

Scripts

Settings

1

```
{ "id": 1 }
```

{ }

e.g.

Use Example Message

Response

Metadata (2)

Trailers

Test results

Status code: 0 OK Time: 65 ms

⇒

1

2

3

4

```
{
  "id": 1,
  "name": "Bob"
}
```

grpc://127.0.0.1:50051

↕ StudentsService / GetAllStudents

Message

Authorization

Metadata

Service definition

Scripts

Settings

1

Compose message

{ }

e.g.

Use Example Message

Response

Metadata (2)

Trailers

Test results

Status code: 0 OK Time: 86 ms  S

⇒

1

2

3

4

5

6

7

8

```
{
  "students": [
    {
      "id": 1,
      "name": "Bob"
    }
  ]
}
```

Step4: Complete the client code.

```
const grpc = require('@grpc/grpc-js');
const protoLoader = require('@grpc/proto-loader');
const packageDefinition = protoLoader.loadSync('students.proto', {});
const StudentsServicePackage =
  grpc.loadPackageDefinition(packageDefinition).StudentsServicePackage;
```

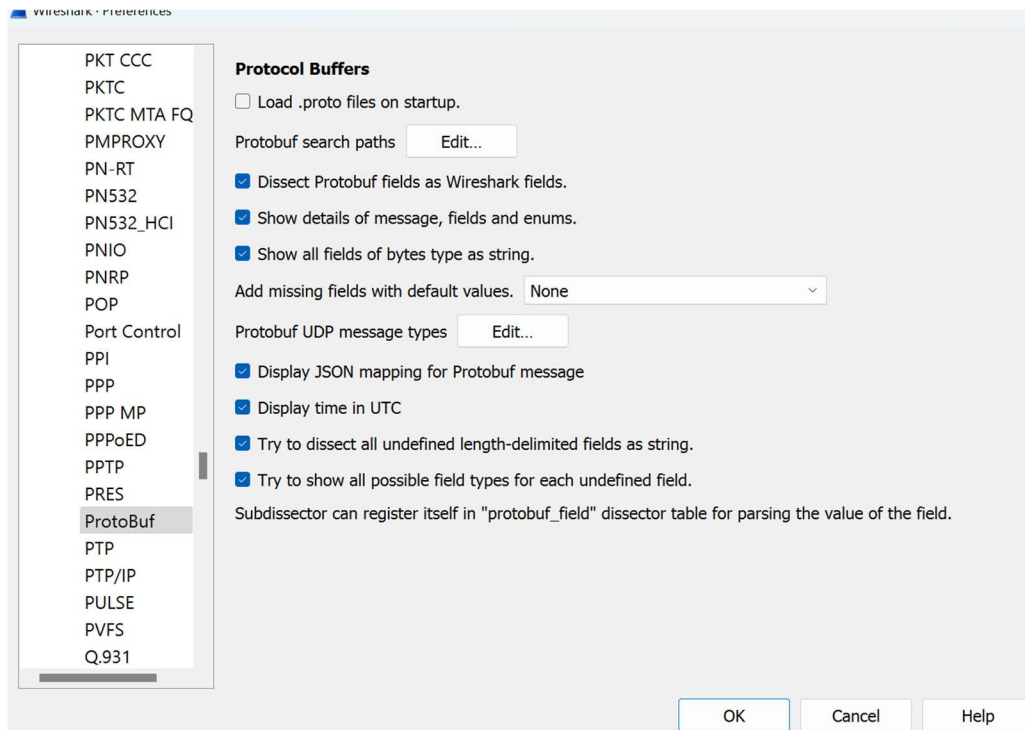
```
const client = new StudentsServicePackage.StudentsService('localhost:50051',
grpc.credentials.createInsecure());

client.addStudent({ 'id': -1, 'name': 'Alice' }, (err, response) => {
...

```

Step5: Analyze gRPC messages using Wireshark.

Tell Wireshark where to find .proto files by setting the Protobuf Search Paths in the preferences accessible from the Edit menu under Preferences > Protocols > Protobuf.



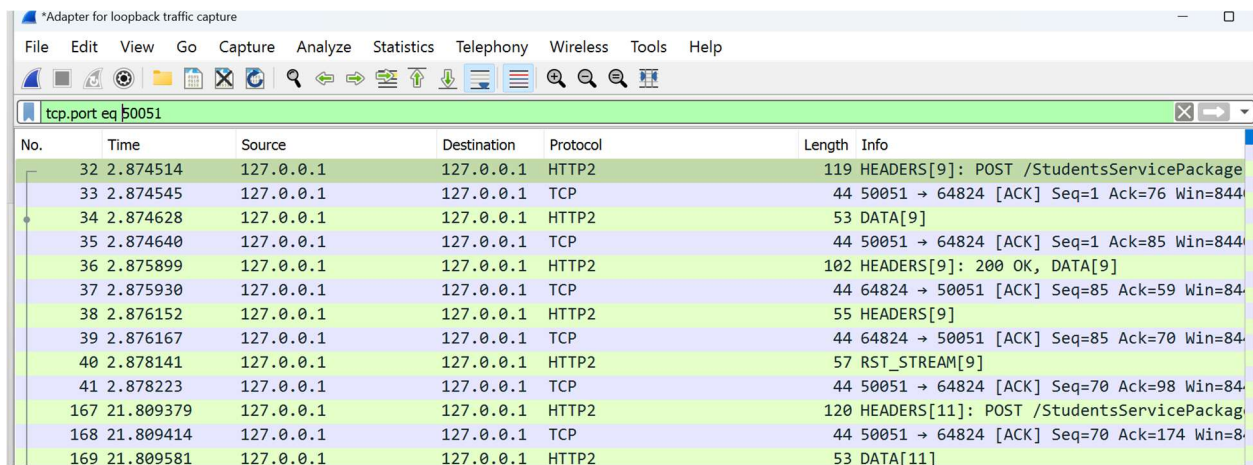
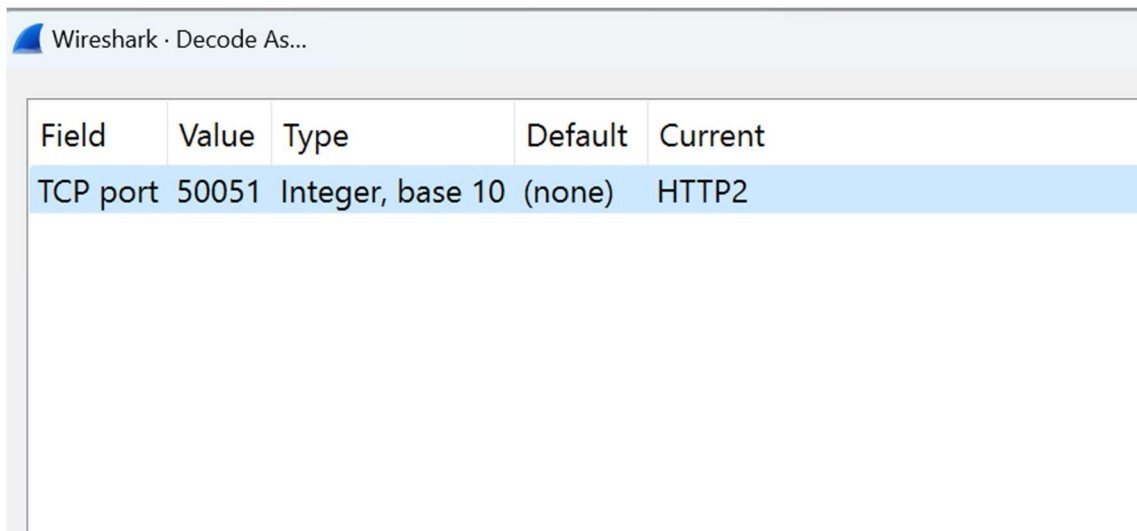
Adapter for loopback traffic capture

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

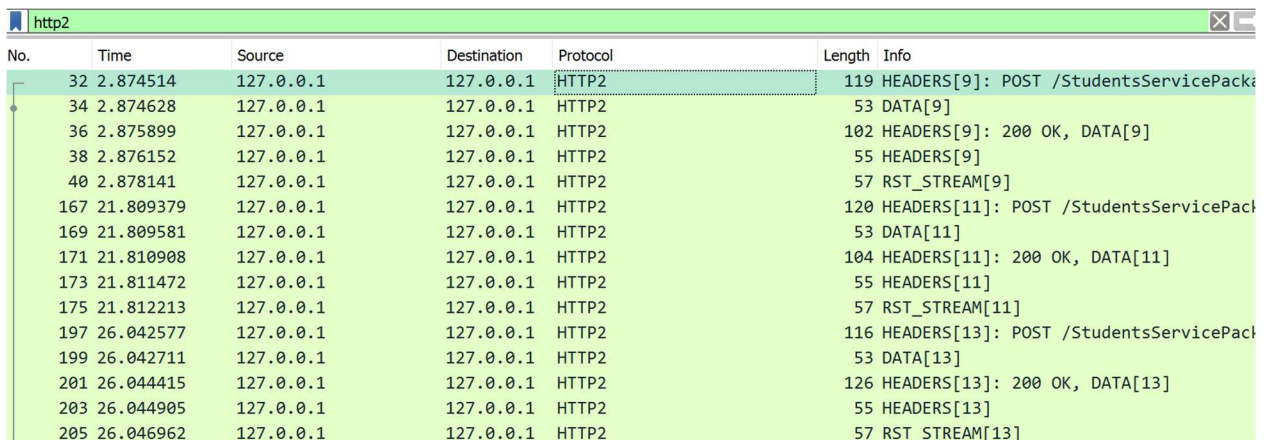
tcp.port eq 50051

No.	Time	Source	Destination	Protocol	Length	Info
32	2.874514	127.0.0.1	127.0.0.1	TCP	119	64824 → 50051 [PSH, ACK] Seq=1 Ack=1 Win=
33	2.874545	127.0.0.1	127.0.0.1	TCP	44	50051 → 64824 [ACK] Seq=1 Ack=76 Win=844
34	2.874628	127.0.0.1	127.0.0.1	TCP	53	64824 → 50051 [PSH, ACK] Seq=76 Ack=1 Wi
35	2.874640	127.0.0.1	127.0.0.1	TCP	44	50051 → 64824 [ACK] Seq=1 Ack=85 Win=844
36	2.875899	127.0.0.1	127.0.0.1	TCP	102	50051 → 64824 [PSH, ACK] Seq=1 Ack=85 Wi
37	2.875930	127.0.0.1	127.0.0.1	TCP	44	64824 → 50051 [ACK] Seq=85 Ack=59 Win=84
38	2.876152	127.0.0.1	127.0.0.1	TCP	55	50051 → 64824 [PSH, ACK] Seq=59 Ack=85 W
39	2.876167	127.0.0.1	127.0.0.1	TCP	44	64824 → 50051 [ACK] Seq=85 Ack=70 Win=84
40	2.878141	127.0.0.1	127.0.0.1	TCP	57	64824 → 50051 [PSH, ACK] Seq=85 Ack=70 W
41	2.878223	127.0.0.1	127.0.0.1	TCP	44	50051 → 64824 [ACK] Seq=70 Ack=98 Win=84
167	21.809379	127.0.0.1	127.0.0.1	TCP	120	64824 → 50051 [PSH, ACK] Seq=98 Ack=70 W
168	21.809414	127.0.0.1	127.0.0.1	TCP	44	50051 → 64824 [ACK] Seq=70 Ack=174 Win=8
169	21.809581	127.0.0.1	127.0.0.1	TCP	53	64824 → 50051 [PSH, ACK] Seq=174 Ack=70
170	21.809597	127.0.0.1	127.0.0.1	TCP	44	50051 → 64824 [ACK] Seq=70 Ack=183 Win=8
171	21.810998	127.0.0.1	127.0.0.1	TCP	104	50051 → 64824 [PSH, ACK] Seq=70 Ack=183

Tell Wireshark that these ports are carrying HTTP2 traffic, through the Decode As dialog, which you access from the Analyze menu (or right-click on an entry from the packet-list pane).



Filter on http2:



Analyze the packets related to the method addStudent.

Part2 : Message Passing Interface (MPI)

Source: MPI Tutorial for Machine Learning (Part 1/3), Thiwanka Chameera Jayasiri, Jul 17, 2023, <https://medium.com/@thiwankajayasiri/mpi-tutorial-for-machine-learning-part-1-3-6b052abe3f29>

To begin, you'll need to install the MPI library and the Python MPI wrapper, mpi4py. The installation process for the MPI library differs depending on the platform.

A Basic MPI Python Program:

A simple Python script that uses MPI to print out a message from each process.

```
from mpi4py import MPI

def main():
    comm = MPI.COMM_WORLD
    rank = comm.Get_rank()
    print(f"This is process {rank} speaking.")

if __name__ == "__main__":
    main()
```

1. What does the MPI.COMM_WORLD object represent in this code?
2. What is the purpose of the Get_rank() method in the code?
3. What will the output of the print(f"This is process {rank} speaking.") statement be when the code is run in a parallel environment?

To run this program using MPI, use the mpiexec command followed by the -n flag and the number of processes you want to create as follows:

```

        .\td_new\mpilab>pip install mpi4py
Defaulting to user installation because normal site-packages is not writeable
Collecting mpi4py
  Downloading mpi4py-4.0.3-cp313-cp313-win_amd64.whl.metadata (10 kB)
    Downloading mpi4py-4.0.3-cp313-cp313-win_amd64.whl (1.7 MB)
      1.7/1.7 MB 32.4 MB/s eta 0:00:00
Installing collected packages: mpi4py
Successfully installed mpi4py-4.0.3

[notice] A new release of pip is available: 24.3.1 -> 25.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip

```

```

        .\td_new\mpilab>mpiexec -n 4 python mpi_example.py
This is process 3 speaking.
This is process 1 speaking.
This is process 0 speaking.
This is process 2 speaking.

```

Distributing Data Preprocessing Tasks

We'll start by generating sample data:

```

generate_data.py X
generate_data.py > ...
1  # generate_data.py
2
3  import numpy as np
4  np.random.seed(0)
5
6  # Generate 10 arrays of shape (5000, 10) and save them as .npy files
7  for i in range(10):
8      data = np.random.randn(5000, 10)
9      np.save(f'data_{i}.npy', data)

```

Write the MPI Python script that will distribute the data on several process for preprocessing tasks like standardizing the data.

Formula for Standardization:

The standard score (z-score) of a feature is calculated as:

$$z = \frac{X - \mu}{\sigma}$$

Where:

- X is an individual data point.
- μ is the mean of the feature.
- σ is the standard deviation of the feature.

Good Luck